Smart Contract Vulnerability Rectification Engine

Report ID	cb364b04-ab6e-479e-8b86-…
Contract	VulnerableDemo.sol
Language	Solidity
Analysis Date	2026-10-22 17:53:18
Generated	2026-10-22 17:53 UTC
Rectification	Not Fixed

Executive Summary

This report presents the results of an automated security analysis of the smart contract **VulnerableDemo.sol** written in **Solidity**. The analysis was conducted by the SCRE automated pipeline on 2026-10-22. A total of **4 vulnerabilities** were identified. The overall rectification status is **Not Fixed**.

Severity	Count	Assessment
Critical	1	Immediate action required
High	1	Fix before deployment
Medium	1	Review and mitigate
Low	1	Monitor and document
Total	**4**	

Contract Information

Filename	VulnerableDemo.sol
Language	Solidity
Analysis Tool	SCRE v1.0 — Pattern-based detector
Detection Method	Reentrancy pattern (low-level .call before state update)
Repair Strategy	Checks-Effects-Interactions + reentrancy lock
Compilation Check	Solidity 0.8.30 via npx solc

Original Source Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

/// @title VulnerableDemo
/// @notice INTENTIONALLY VULNERABLE training contract for local demo/scanner testing only.
/// @dev Do not deploy this contract to any public or production network.
contract VulnerableDemo {
    address public owner;
    address public implementation;
    uint256 public totalDeposits;
    uint256 public lastActionTime;
    uint256 public unusedCounter; // Unused state variable (code-quality finding)

    mapping(address => uint256) public balances;
    address[] public participants;
    mapping(address => bool) private knownParticipant;

    event Deposited(address indexed user, uint256 amount);
    event Withdrawn(address indexed user, uint256 amount);

    constructor() {
        owner = msg.sender;
    }

    receive() external payable {
        deposit();
    }

    function deposit() public payable {
        require(msg.value > 0, "Deposit must be positive");
        balances[msg.sender] += msg.value;
        totalDeposits += msg.value;
        if (!knownParticipant[msg.sender]) {
            knownParticipant[msg.sender] = true;
            participants.push(msg.sender);
        }
        emit Deposited(msg.sender, msg.value);
    }

    /// FINDING 1: Reentrancy risk — external call occurs before balance update.
    /// FINDING 2: Unchecked low-level call return value.
    function withdraw(uint256 amount) external {
```

```
        require(balances[msg.sender] >= amount, "Insufficient balance");
        (bool success, ) = payable(msg.sender).call{value: amount}("");
        // Intentionally ignores `success`.
        balances[msg.sender] -= amount;
        totalDeposits -= amount;
        emit Withdrawn(msg.sender, amount);
    }

    /// FINDING 3: Missing access control — anyone can change the owner.
    function changeOwner(address newOwner) external {
        owner = newOwner;
    }

    /// FINDING 4: tx.origin used for authorization instead of msg.sender.
    function ownerWithd
… [truncated for report]
```

Vulnerability Analysis

Reentrancy Vulnerability [CRITICAL] — Line 8

Description: External call before state update allows reentrancy attack.

Impact: Complete drain of contract ETH balance via recursive calls.

Mitigation: Apply Checks-Effects-Interactions pattern; use ReentrancyGuard.

Unchecked Call Return Value [HIGH] — Line 9

Description: Return value of low-level .call() is not validated.

Impact: Silent failures — failed transfers go unnoticed.

Mitigation: Always check the bool return value; prefer transfer() or revert on failure.

Missing Access Control [MEDIUM] — Line 9

Description: withdraw() can be called by any external account.

Impact: Unauthorised users may trigger withdrawals.

Mitigation: Add an onlyOwner modifier or role-based access control.

Integer Overflow/Underflow (Potential) [LOW] — Line 7

Description: No SafeMath or built-in overflow checks for older Solidity versions.

Impact: Arithmetic wrap-around may cause unexpected balance manipulation.

Mitigation: Use Solidity ≥0.8.0 (built-in checks) or OpenZeppelin SafeMath.

Suggested Rectifications

Rectification was not applied: Unsupported contract: expected VulnerableBank.

Manual review and remediation by a qualified Solidity developer is recommended.

Security Recommendations

• **Apply Checks-Effects-Interactions (CEI):** Always update contract state before making external calls.

• **Use ReentrancyGuard:** Consider OpenZeppelin's battle-tested ReentrancyGuard modifier for all withdrawal functions.

• **Validate all external call return values:** Never ignore the boolean returned by .call().

• **Implement access control:** Use Ownable or role-based patterns for sensitive functions.

• **Use Solidity ≥0.8.0:** Built-in overflow/underflow protection removes an entire class of arithmetic vulnerabilities.

• **Conduct manual code review:** Automated tools detect known patterns; novel vulnerabilities require expert human review.

• **Deploy to testnet first:** Always test thoroughly on a public testnet before mainnet deployment.

• **Consider a professional security audit:** For any contract handling real funds, engage a reputable auditing firm.

Disclaimer

This report was generated by the SCRE automated analysis prototype. The analysis is pattern-based and limited to a specific subset of known vulnerability classes. A clean result or successful compilation does not guarantee the complete absence of vulnerabilities. Generated repairs must be reviewed, tested, and validated by a qualified Solidity developer before use. SCRE is provided for educational and research purposes only. Do not deploy any contract — repaired or otherwise — to mainnet based solely on this report. The authors accept no liability for damages arising from the use of this tool or its output.